



TRIBUBHAN UNIVERSITY

Institute of Science and Technology (IoST)

Project Report on

**“EthnoArchive: Traditional Musical
Instrument Classification System.”**

Submitted To

National College of Computer Studies

**Department of Bachelor of Science in Computer Science and
Information Technology (BsC.CSIT)**

*In partial fulfillment of the requirements for Bachelor's in
Computer Science and Information Technology (BSC.CSIT)*

Submitted By

Niraj Maharjan [28876/078]

Regd. No.: 5-2-551-80-2021

Under the supervision of

Mr Radha Krishna Gajurel

Date: September 18, 2025

SUPERVISOR’S RECOMMENDATION

This is to recommend that **Niraj Maharjan**, (28876, 5-2-551-80-2021), has carried out project work entitled “**EthnoArchive: Traditional Musical Instrument Classification System**”) for the requirement to the project work in Bachelor of Science (B.Sc.) in Computer Science and Information Technology under my supervision in National College of Computer Studies, Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal.

To my knowledge, this work has not been submitted for any other degree.

He has fulfilled all the requirements laid down by the Institute of Science and Technology (IoST), Tribhuvan University (T.U.), Nepal for the submission of the project work for the partial fulfillment of Bachelor of Science (B.Sc.) degree.

Mr. Radha Krishna Garjurel

College: National College of Computer Studies

University: Tribhuvan Unversity (T.U.)

[18/09/2025]

Date: 03/09/2025

CERTIFICATE OF APPROVAL

This is to certify that the report is evaluated and recommended to the Department of Computer Science and Information Technology for acceptance of the report entitled **“ETHNOARCHIVE: TRADITIONAL MUSICAL INSTRUMENT CLASSIFICATION STSTEM”** submitted by **“NIRAJ MAHARJAN”** in partial fulfillment for the degree of Bachelor of Science in Computer Science and Information Technology (B.Sc.CSIT), Institute of Science and Technology (IOST), Tribhuvan University.

SIGNATURE OF SUPERVISOR Mr. Radha Kishna Gajurel	SIGNATURE OF COORDINATOR Mr. Rajan Paudel National College of Computer Studies
SIGNATURE OF INTERNAL EXAMINER	SIGNATURE OF EXTERNAL EXAMINER

ACKKNOWLEDGEMENT

It gives me immense pleasure to express our sincere gratitude with the happiest appreciation to all those respectable personalities who helped me to make my project work successive as well as productive. My sincere appreciation goes to my mentors, teachers, and guides, without whose unwavering support and assistance this project would not have come to fruition. I'm deeply thankful for the continuous help and support provided by the college throughout the development of the project. Special thanks are owed to Mr. Radha Krishna Gajurel for his invaluable supervision and assistance. His guidance and support were crucial during challenging moments, and without him, this project would not have been possible. I'm grateful for his consistent error corrections, valuable suggestions and guidance that provided to instrumental throughout the project development. Additionally, I extend my thanks to all the teachers who provided the opportunity to undertake this project. Their encouragement and support were integral to the project's growth and success.

ABSTRACT

EthnoArchive is a digital system designed to preserve Nepal's cultural heritage by classifying traditional musical instruments such as Bansuri, Khin, Madal, Dhimey, and Sarangi. The problem addressed is the lack of structured archives and the difficulty of identifying instruments in scattered, mislabeled digital content. The objective was to build an automated, user-friendly web platform that enables audio uploads, accurate instrument tagging, and cultural context. Using the Waterfall methodology, the project progressed through systematic phases, applying CNN with mel-spectrogram feature extraction for classification. The system, developed with React, Django, and PostgreSQL, achieved 93% accuracy, demonstrating strong potential for cultural documentation and education. This project bridges the gap between tradition and technology, creating a foundation for future improvements in real-time recognition and larger-scale digital heritage preservation.

Keywords: *EthnoArchive, traditional instruments, Nepali music, CNN, mel-spectrogram, audio classification, digital archive, machine learning, cultural preservation, Django, React, PostgreSQL*

TABLE OF CONTENTS

SUPERVISOR’S RECOMMENDATION	ii
CERTIFICATE OF APPROVAL	iii
ACKKNOWLEDGEMENT	iv
ABSTRACT.....	v
LIST OF FIGURES	viii
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
CHAPTER 1: INTRODUCTION.....	1
1.1 Introduction	1
1.2 Problem Statement	1
1.3 Objectives.....	2
1.4 Scope and Limitations.....	2
1.4.1 Scope.....	2
1.4.2 Limitations	2
1.5 Methodology	2
1.6 Report Organization	5
CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW	7
2.1 Background Study.....	7
2.2 Literature Review.....	9
CHAPTER 3: SYSTEM ANALYSIS	11
3.1 System Analysis.....	11
3.1.1 Requirement Analysis.....	11
3.1.2 Feasibility Study	13
3.1.3 Analysis.....	16
CHAPTER 4: SYSTEM DESIGN	20

4.1 Design	20
4.1.1 System Architecture.....	20
4.1.2 Deployment Diagram.....	21
4.2 Algorithm Description	22
4.2.1 Fast Fourier Transform (FFT) - Cooley-Tukey Algorithm.....	22
4.2.2 STFT	26
CHAPTER 5: IMPLEMENTATION AND TESTING	29
5.1 Implementation	29
5.1.1 Tools Used	29
5.1.2 Data Preprocessing.....	30
5.1.3 Data Preparation.....	34
5.1.4 Model Architecture	35
5.2 Testing.....	38
5.3 Result Analysis	42
CHAPTER 6: CONCLUSION AND FUTURE RECOMMENDATIONS	45
6.1 Conclusion	45
6.2 Future Recommendations	45
REFERENCE.....	46

LIST OF FIGURES

Figure 1. 1: Waterfall model phases	5
Figure 3. 1: Use Case Diagram of EthnoArchive	12
Figure 3. 2: Gantt Chart for phases of EthnoArchive	16
Figure 3. 3: Class Diagram of EthnoArchive	16
Figure 3. 4: Sequence Diagram of EthnoArchive	17
Figure 3. 5: Activity Diagram of EthnoArchive	18
Figure 4. 1: System Architecture of EthnoArchive	20
Figure 4. 2: Deployment Diagram of EthnoArchive	21
Figure 4. 3: Manual FFT Algorithm Implementation	25
Figure 4. 4: Hann window implementation	28
Figure 4. 5: Custom STFT Implementation	28
Figure 5. 1: File Validation Implemention	30
Figure 5. 2: Audio Format Standardization Implementation	31
Figure 5. 3: Audio Normalization Implementation	31
Figure 5. 4: Chunking Implementation	32
Figure 5. 5: Mel Spectrogram Extraction Implementation	32
Figure 5. 6: Sarangi Mel Spectrogram Visualization	33
Figure 5. 7: Dhimey Mel Spectrogram Visualization	33
Figure 5. 8: Bansuri Mel Spectrogram Visualization	333
Figure 5. 9: CSV File Format Demo	34
Figure 5. 10: Data Split Summary	34
Figure 5. 11: Classwise Distribtuition Summary	35
Figure 5. 12: Model Architecture	37
Figure 5. 13: Classification Report	42
Figure 5. 14: Confusion Matrix	43

Figure 5. 15 Error Rate by Class.....	44
--	-----------

LIST OF TABLES

Table 3. 1 Hardware Requirements.....	13
Table 3. 2 Software Requirements	13
Table 3. 3 Schedule Plan.....	15
Table 5. 1 Test Cases for authentication module	38
Table 5. 2 Test Cases for upload and processing module	39
Table 5. 3 Instrument Detection Module	40

LIST OF ABBREVIATIONS

ADC	Analog Digital Conversion
API	Application Programming Interface
CNN	Convolutional Neural Network
CSV	Comma Separated Values
DFT	Discrete Fourier Transform
ER	Entity Relationship
FFT	Fast Fourier Transform
LSTM	Long Short-Term Memory
ML	Machine Learning
PCM	Pulse Code Modulation
RNN	Recurrent Neural Network
STFT	Short Time Fourier Transform

CHAPTER 1

INTRODUCTION

1.1 Introduction

Traditional music forms the backbone of any culture, and Nepal's rich musical heritage is no exception. For centuries, instruments like the bansuri, madal, sarangi, dhimey, and khin have been central to Nepali folk music, religious ceremonies, and cultural celebrations. These instruments carry stories, emotions, and traditions that have been passed down through generations.

However, in our rapidly digitalizing world, this musical knowledge faces a real threat. Many young Nepalese, especially those living in urban areas or abroad, struggle to identify these traditional instruments when they hear them. The older generation of musicians who are the keepers of this knowledge won't be around forever, and there's a growing gap between traditional culture and modern life.

At the same time, we're seeing an explosion of digital content featuring these instruments on platforms like YouTube, TikTok, and various streaming services. While this might seem like preservation, the reality is that this content is scattered, unorganized, and often mislabeled. A video titled "beautiful flute music" might actually feature a bansuri, but viewers don't learn the proper name or cultural context of what they're hearing.

This is where technology can step in to help. Machine learning has shown remarkable success in audio classification tasks, from identifying speech to recognizing different genres of music. The question we're asking is: can we use these same techniques to automatically identify and catalog traditional Nepali instruments?

1.2 Problem Statement

The lack of structured labeling in traditional music archives hinders the search, analysis, and preservation of recordings by instrument. Existing classifiers focus on Western or classical instruments, leaving traditional Nepali instruments like bansuri and madal underrepresented. Audio files often lack accurate metadata, and manual classification is slow, error-prone, and impractical for large datasets, especially with mixed recordings. This project addresses the urgent need for an automated system to identify multiple traditional instruments, supporting cultural documentation and education.

1.3 Objectives

- To build a user-friendly web application that allows to upload audio, receive instrument tags, and learn about their cultural significance.
- To build a machine learning model, using CNNs and mel spectrograms, to identify and tag traditional instruments in user-uploaded audio clips.

1.4 Scope and Limitations

1.4.1 Scope

- The system will focus on the classification of a defined set of Newari instruments: bansuri, dhimey, khin, madal and sarangi.
- Users will be able to upload audio clips from traditional Newari music.
- The project will utilize supervised learning approach, CNNs, for audio tagging.
- The output will be integrated into an archive system for improved accessibility and searchability.
- Feature extraction will be based on established techniques such as mel spectrograms.

1.4.2 Limitations

- The system's performance is dependent on the quality and diversity of the training dataset; rare instrument combinations or noisy recordings may reduce accuracy.
- Only a limited set of instruments are covered; the system will not generalize to other instruments.
- Real-time tagging or deployment on resource-constrained devices is outside the current project's scope.

1.5 Methodology

This project adopted the Waterfall software development methodology due to the structured nature of the research objectives and the clearly defined requirements for cultural heritage preservation. The waterfall approach was particularly suitable for this academic project as it provided systematic progression through distinct phases with well-documented deliverables at each stage. The waterfall methodology was selected based on several key factors:

- **Clear Requirements:** The project had well-defined objectives from the outset - developing a classification system for five specific traditional Nepali instruments
- **Academic Timeline:** The structured phases aligned well with academic project timelines and milestone requirements
- **Sequential Dependencies:** Each phase built upon the previous one, particularly the data collection feeding into model training, which then informed system development
- **Documentation Requirements:** The methodology facilitated comprehensive documentation at each stage, essential for academic research

Development Phases:

Phase 1: Requirements Analysis and Planning

Objectives: Define system requirements, establish project scope, and create development roadmap

Activities:

- Literature review of audio classification techniques and cultural preservation projects
- Technical feasibility analysis for CNN-based audio classification
- System requirements specification development
- Project timeline and resource allocation planning

Phase 2: System Design

Objectives: Create comprehensive system architecture and design specifications

Activities:

- Database schema design
- CNN model architecture planning and mel spectrogram feature extraction pipeline design
- Web application architecture specification (React frontend, Django backend)
- User interface mockups and workflow design
- API specification for frontend-backend communication

Phase 3: Data Collection and Preprocessing

Objectives: Gather and prepare training dataset for machine learning model

Activities:

- Audio data collection from YouTube and TikTok platforms and kaggle
- Organization of recording sessions with musicians
- Format standardization and audio length normalization to 10-second clips
- Manual labeling and verification of instrument classifications
- Mel spectrogram feature extraction from processed audio files
- Dataset splitting into training, validation, and test sets (70:15:15 ratio)

Phase 4: Machine Learning Model Development

Objectives: Develop and train CNN classification model

Activities:

- CNN architecture implementation using TensorFlow framework
- Model training with extracted mel spectrograms and labeled dataset
- Performance optimization and overfitting prevention techniques
- Model evaluation using confusion matrices and accuracy metrics

Phase 5: Backend Development

Objectives: Implement server-side functionality and model integration

Activities:

- Django project setup and configuration
- PostgreSQL database implementation based on design specifications
- User authentication and session management system development
- File upload and processing API endpoints creation
- Machine learning model integration with web backend
- Audio preprocessing pipeline implementation in production environment

Phase 6: Frontend Development

Objectives: Create user-friendly web interface

Activities:

- React application setup and component architecture implementation
- User interface development based on approved mockups
- Drag-and-drop file upload functionality creation

- Classification results visualization and display components

Phase 7: System Integration and Testing

Objectives: Integrate all components and conduct comprehensive testing

Activities:

- End-to-end system integration and connectivity testing
- Bug identification and resolution processes

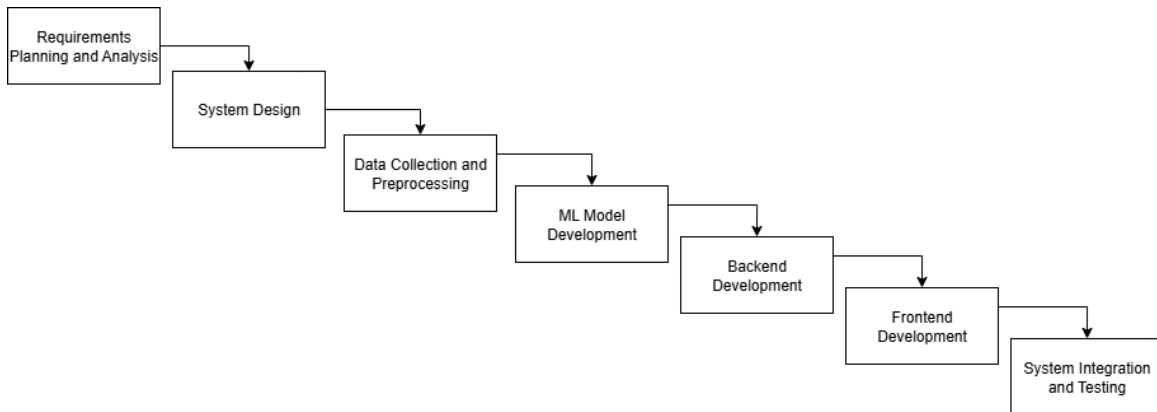


Figure 1. 1: Waterfall model phases

1.6 Report Organization

This report has been prepared following the guidelines provided by Tibhuvan Univeristy. The report is separated into different chapters and each consists of several sub-chapters. The foundational section of the report consists of Title Page, Acknowledgement, Abstract, Table of Contents, List of Abbreviations, List of Figures and List of Tables. The main report is divided into six groups, which include:

Chapter 1: Introduction

It includes the general overview of the system and the project as a whole. It includes the Problem Statement, Objectives, Scope/Limitations and the Development Methodologies for the project and the system being developed.

Chapter 2: Background Study and Literature Review

It includes the study of the current scenario; the system will be deployed into. It includes the study of current trends, preferences of people, the existing system, and description of

fundamental theories, general concepts and terminologies related to the project, review of the similar/relevant projects, theories and results by other researchers.

Chapter 3: System Analysis

It includes the requirement and feasibility analysis of the system that can be generated through the studies presented in the previous two chapters. It will also include the flowchart and ER Diagram for the system which specifies the workflow, entities, attributes and their relationships.

Chapter 4: System Design

It includes the design of database, forms and interface of the system. It also includes the implementation details of the selected methodology and the details of the algorithms used.

Chapter 5: Implementation and Testing

It includes the details of the different design and development tools used and the implementation details of the modules presented in the form of code snippets of functions classes. It also includes the testing of the system with different test cases as per the requirements.

Chapter 6: Conclusion and Future Recommendations

It includes the summary of the system and the project as a whole. It also includes the possibilities/aspects which the system can implement in the future.

The final part of the report consists of References and Appendices. The references are listed in accordance to the IEEE referencing standards. The appendices includes the screenshots of the system and the major code snippets.

CHAPTER 2

BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Audio signals in their raw form are continuous analog waveforms representing variations in air pressure over time. To process these signals digitally, they must first be converted through ADC, where the continuous signal is sampled at regular intervals and quantized into discrete digital values. For this project, working with 10-second audio clips sampled at 22.05 kHz provides sufficient temporal resolution and frequency bandwidth to capture the essential characteristics of traditional Newari instruments.

The fundamental properties of audio signals include amplitude (representing loudness), frequency (representing pitch), and phase (representing timing relationships). These properties are analyzed in both time and frequency domains to extract meaningful features for classification. Time-domain analysis examines how the signal's amplitude varies over time, revealing information about the signal's envelope and transient characteristics. Frequency-domain analysis, typically achieved through the Fourier Transform, reveals the spectral content of the signal, showing how energy is distributed across different frequencies.

STFT provides a bridge between time and frequency analysis by applying the Fourier Transform to overlapping segments of the audio signal. This technique creates a time-frequency representation called a spectrogram, which visualizes how the frequency content of an audio signal changes over time. Spectrograms serve as the foundation for many advanced audio processing techniques and are particularly valuable for analyzing musical instruments, as they can reveal the harmonic structure and temporal evolution of different instrumental sounds.

Spectral Feature Extraction Techniques

Spectral features capture the frequency characteristics of audio signals and are fundamental to instrument classification systems. These features provide insights into the harmonic content, timbral qualities, and spectral distribution of musical sounds.

Mel spectrograms are a widely adopted feature extraction technique in audio processing, particularly for tasks like music and speech analysis. They provide a robust representation

of the audio signal by capturing its spectral characteristics in a way that aligns with human auditory perception. Unlike raw audio waveforms, Mel spectrograms transform the signal into a time-frequency representation that emphasizes perceptually relevant features, making them well-suited for machine learning tasks such as instrument classification.

The process of generating a Mel spectrogram involves several key steps. First, the audio signal is divided into short, overlapping time frames, and STFT is applied to each frame to obtain the power spectrum. Second, the frequency bins of the power spectrum are mapped to the Mel scale using triangular overlapping filters. The Mel scale is logarithmic, mimicking human perception of pitch, which is non-linear across the frequency spectrum. Third, the power values are typically converted to a logarithmic scale to better reflect human auditory sensitivity. The resulting Mel spectrogram is a two-dimensional representation, with time on one axis and Mel-scaled frequency on the other, providing a compact yet detailed view of the signal's spectral content over time. This rich representation captures both the frequency content and its temporal evolution, making it highly effective for distinguishing the unique timbral characteristics of instruments like the Bansuri, Khin, Madal, Dhimey, and Sarangi in the context of this project.

Once features are extracted, classification models are trained to recognize patterns. Deep learning models, particularly CNNs, have become the foundation for audio classification. CNNs are neural networks designed for grid-like data (e.g. images), using convolutional filters to learn local patterns. In audio tasks, spectrograms serve as image-like inputs to CNNs, allowing the network to autonomously learn hierarchical time–frequency features. Pooling layers then reduce dimensions and prevent overfitting, while nonlinear activations (e.g. ReLU) allow modeling of complex patterns. This end-to-end approach has proven highly effective: CNNs can learn both low-level spectral textures and higher-level timbral patterns, outperforming traditional shallow classifiers in many cases.

Other deep architectures include RNNs and hybrids. RNNs (such as LSTM) process sequences of features and can capture temporal dependencies, which is useful for continuous audio classification. However, RNNs often require more training time and are less parallelizable than CNNs on spectrogram images. In practice, many modern instrument-classification systems rely on CNN-based models (or combined CNN–RNN hybrids) applied to spectrogram inputs because of their efficiency and strong performance.

2.2 Literature Review

The field of musical instrument classification has witnessed significant advancement through the application of various machine learning techniques, with researchers exploring different approaches to feature extraction, model architecture, and evaluation methodologies.

Chulev conducted a comprehensive study applying various machine learning methods to musical instrument classification [1]. The research evaluated multiple approaches including Naive Bayes, Support Vector Machines, Random Forests, boosting techniques like AdaBoost and XGBoost, as well as deep learning models such as CNNs and artificial neural networks. Using the NSynth dataset, the study demonstrated the relative effectiveness of different approaches and highlighted the advantages and limitations of each method. The research particularly emphasized the importance of feature engineering and the potential benefits of hybrid modeling approaches, providing valuable guidance for researchers working on similar classification tasks.

Haidar-Ahmad presented a CNN-based approach for multi-class instrument classification focusing on piano, drums, flute, and other categories [2]. The study utilized mel-spectrograms as input features and achieved precision of 70%, recall of 65%, and F1-score of 64%. This work demonstrated the effectiveness of CNN architectures for instrument classification while also highlighting the challenges associated with limited training data and the need for robust feature representations. The research provided valuable insights into the preprocessing steps required for effective spectrogram-based classification.

The application of deep learning techniques to audio classification has revolutionized the field, with CNNs proving particularly effective for processing spectro-temporal representations of audio signals.

Hing and Settle presents an innovative approach to classify musical instruments using CNNs. Highlighting the growing need for instrument identification in digital music formats, the authors develop a multi-class instrument classifier that leverages transfer learning. Their model effectively categorizes instruments within audio clips, achieving notable results through a robust architecture consisting of four convolutional layers and two fully connected layers. [3]

Liu investigated CNN-based approaches for audio feature extraction and classification across diverse applications including speech recognition, music genre classification, and

environmental sound monitoring [4]. The study demonstrated high accuracy, precision, recall, and F1-scores across various audio processing tasks, with comparative analysis showing the superiority of CNNs over traditional machine learning methods. The research addressed challenges such as dataset annotation, computational complexity, and robustness to noise, providing practical insights for implementing CNN-based audio processing systems.

The development of automated audio classification systems has significant implications for audio archiving and music information retrieval applications, contributing to the preservation and accessibility of cultural heritage.

Damnjanovic et al addressed the challenges of enabling access to sound archives through integration, enrichment, and retrieval [5]. Their EASAIER system featured enhanced cross-media retrieval functionality, multimedia synchronization, and audio processing tools combined within a configurable interface. The research emphasized the importance of systematic approaches to audio archive management and highlighted the potential for automated classification systems to improve archive accessibility and usability.

Deng et al. provided a comprehensive examination of feature extraction and selection for musical instrument classification, emphasizing the importance of identifying a compact and effective set of features. The study utilized various feature schemes, including human perception-based features, cepstral features, and MPEG-7 audio descriptors. Their empirical findings highlighted the redundancy present within and between feature schemes, underscoring the necessity for further research to optimize feature selection methodologies [6].

This comprehensive literature review demonstrates the substantial progress that has been made in audio classification and musical instrument recognition while highlighting the specific challenges and opportunities relevant to developing automated tagging systems for traditional instruments.

CHAPTER 3

SYSTEM ANALYSIS

3.1 System Analysis

System analysis is the critical step for the traditional musical instrument classification system. This system is designed to allow people upload the audio files and archive it for the preservation of the culture.

3.1.1 Requirement Analysis

The requirement analysis will outline what the system needs to accomplish the purpose of musical instrument classification. This includes both functional and non-functional requirements and describe how the system operates overall.

i. Functional Requirements

The functional requirements describe what the system should do and focuses on the behavior and features of the system. As for this system, the functional requirements include registration and login, upload audio files, and view the results of classification.

Use Case Diagram

The use case diagram for the Traditional Musical Instrument Classification System details the functional requirements for two key actors, User and Admin. For the User, the system provides functionalities such as register, allowing new users to create accounts with essential details, and login, a prerequisite for accessing other features. Users can upload audio file, which includes the sub-use case validate audio file to check file format and size before processing. Additionally, users can view result to see ML classification outcomes and browse archive to explore previously processed audio files. For the Admin, the diagram outlines advanced capabilities including manage dataset for curating training data, retrain model to enhance ML accuracy, monitor system logs for system health tracking, and manage archive to oversee file organization. These use cases collectively ensure a robust user experience for audio classification while equipping admins with comprehensive management tools, all integrated through the frontend, backend, and local database infrastructure.

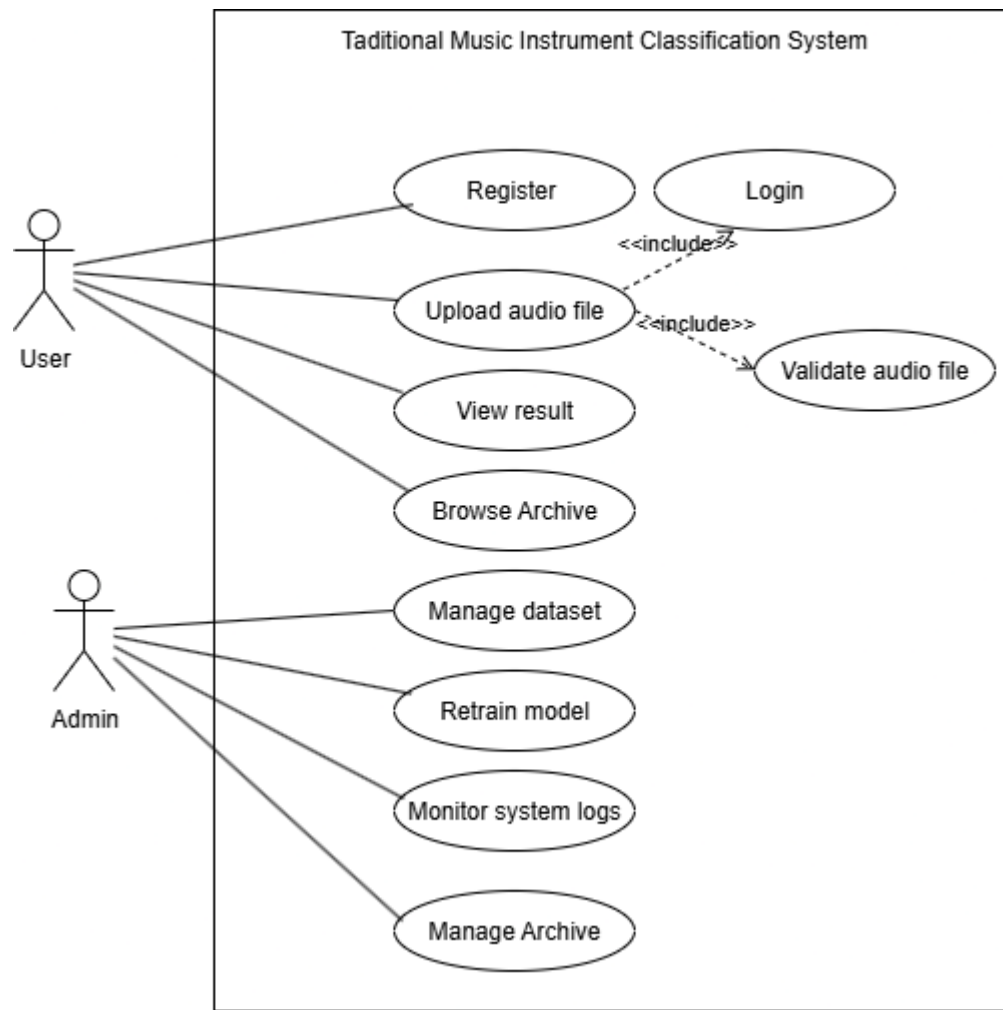


Figure 3. 1: Use Case Diagram of EthnoArchive

ii. Non-Functional Requirements

Non-functional requirements can be defined as the system components where it defines on how the system operate focusing on performance, security, usability and reliability. Some of the non-functional requirements of EthnoArchive are mentioned as below:

- **Accuracy:** The model shall achieve a minimum accuracy of 85% F1-score for instrument identification under clean audio conditions.
- **Performance:** The system shall process and tag audio clips within 5–10 seconds per clip on a standard laptop or cloud platform.
- **Usability:** The web interface shall be intuitive, requiring minimal user training, suitable for students, educators, and cultural enthusiasts.

- Scalability: The dataset shall be extensible to include additional Newari instruments or other Nepali music in future iterations.
- Cultural Sensitivity: Tags and cultural information shall be validated by traditional musicians or experts to ensure authenticity and respect for cultural nuances.

3.1.2 Feasibility Study

This section evaluates the feasibility of the Nepali Traditional Instrument Classification Project across five key dimensions: technical, economic, operational, schedule and legal. The analysis ensures that the project is viable within the constraints of an academic research context while addressing its goal of preserving and cataloging Nepali traditional music through a machine learning-based digital archive.

i. Technical Feasibility

The technical feasibility is concerned with the hardware and software resources that are accessible while building up to the proposed system. Requirements of the system can be categorized as:

Table 3. 1 Hardware Requirements

Device	Laptop/Desktop
Processor	Minimum: Dual-core processor(intel i3) Recommended: Quad core (intel i5 or equivalent)
Storage	At least 8GB of free space for system files and datasets
Network	Stable internet connection for API calls and interactions

Table 3. 2 Software Requirements

Operating System	Windows 10, Linux or MacOS
Frontend Framework	ReactJS
Backend framework	Django
Database	PostgreSQL

API	Django REST Framework
Development Tools	Visual Studio Code
Web Browser	Brave, Chrome etc
Audio Cleaning	Audacity
Model Training	Google Collab

ii. Operational Feasibility

The project is operationally feasible, as it is designed to be user-friendly, maintainable, and aligned with the needs of its target audience. The web application provides a straightforward interface for uploading audio clips, viewing classification results, and accessing the digital archive, requiring minimal technical expertise from users. The Django backend ensures efficient handling of file uploads and database queries, while the PostgreSQL database supports structured storage of audio files and metadata, enabling easy retrieval and management.

From an operational perspective, the system is currently functional in a local environment, with all components - audio preprocessing, model inference, and result display, integrated seamlessly. Challenges such as handling corrupt audio files or varied formats are mitigated through error-handling mechanisms and input validation, ensuring reliable performance.

iii. Economic Feasibility

As an academic research project, the economic feasibility of this initiative is assessed based on minimal financial investment and reliance on open-source or freely available resources. The technology stacks like React, Django, PostgreSQL, TensorFlow is open-source, eliminating licensing costs. Dataset collection from public platforms like YouTube, TikTok, and Kaggle incurs no direct costs, though manual curation and preprocessing require time investment from the researcher.

Development is primarily conducted in a local environment, leveraging existing hardware (e.g., a standard laptop). For training the CNN model, free cloud-based platforms like Google Colab provide sufficient computational power, avoiding the need for expensive hardware.

iv. Legal Feasibility

The system uses open-source datasets or datasets curated by the researcher. Only public or authorized music is used for training purposes. The dataset is sourced from public platforms like YouTube, TikTok, and Kaggle, where content is typically licensed for public use. No personally identifiable information (PII) is stored or processed in the system. So, the system is legally feasible.

v. Schedule Feasibility

The time given for the completion of this project was 3 months. Since the project has machine learning mechanisms, the project took some more time than done usually. Hence, the project has been developed according to the following time schedule to make application schedule feasible.

Table 3. 3 Schedule Plan

SN	Task Name	Duration	Start Date	End Date
1	Requirement Analysis and Planning	8	13 th May	22 nd May
2	System Design	8	23 rd May	3 rd June
3	Data Collection and Preprocessing	17	4 th June	26 th June
4	ML Model Development	17	27 th June	21 st Jul
5	Backend Development	10	22 nd July	8 th August
6	Frontend Development	10	9 th August	18 th August
7	System Integration and Testing	6	19 th August	26 th August

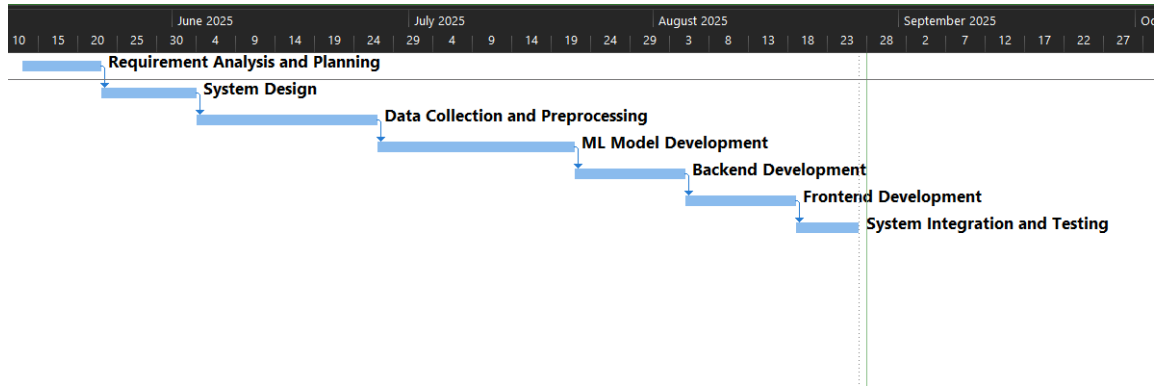


Figure 3. 2: Gantt Chart for phases of EthnoArchive

3.1.3 Analysis

Class Diagram

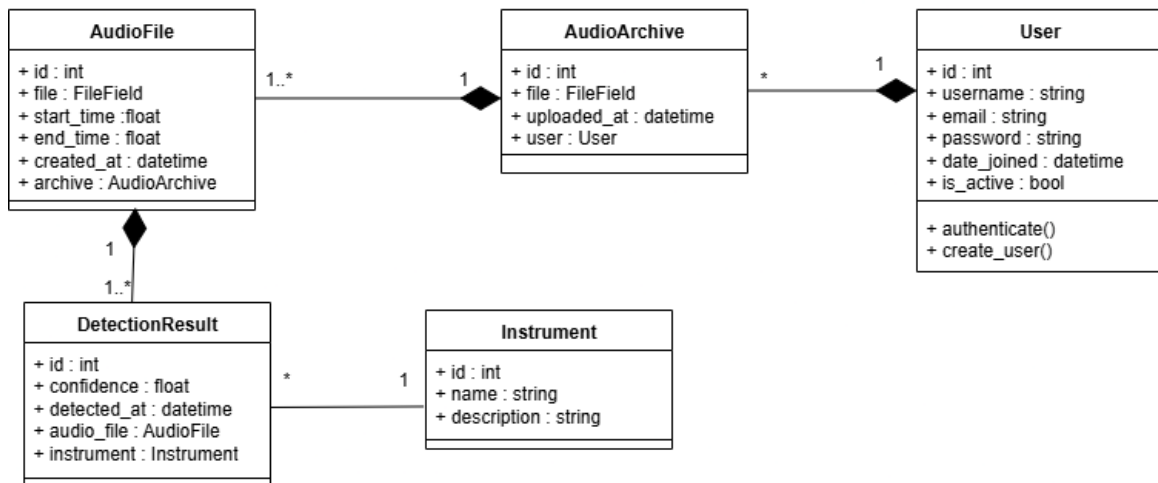


Figure 3. 3: Class Diagram of EthnoArchive

The class diagram represents the data structure of the Nepali Instrument Classification system. A User can upload multiple audio archives, each represented by the AudioArchive class, which stores the full audio clip along with its upload details. Each archive is further divided into smaller segments called AudioFiles, ensuring that clips can be processed in chunks. For every audio file, the system generates one or more DetectionResults, which contain the predicted instrument, the confidence score of the classification, and the timestamp of detection. These results are directly linked to the Instrument class, which stores the list of traditional Nepali instruments along with their descriptions. The relationships in the diagram show that a user can own many archives, each archive can

produce multiple audio files, and each audio file can lead to multiple detection results, while each detection result is tied to one instrument. This structure ensures efficient storage, processing, and retrieval of audio data along with accurate classification of instruments.

Sequence Diagram

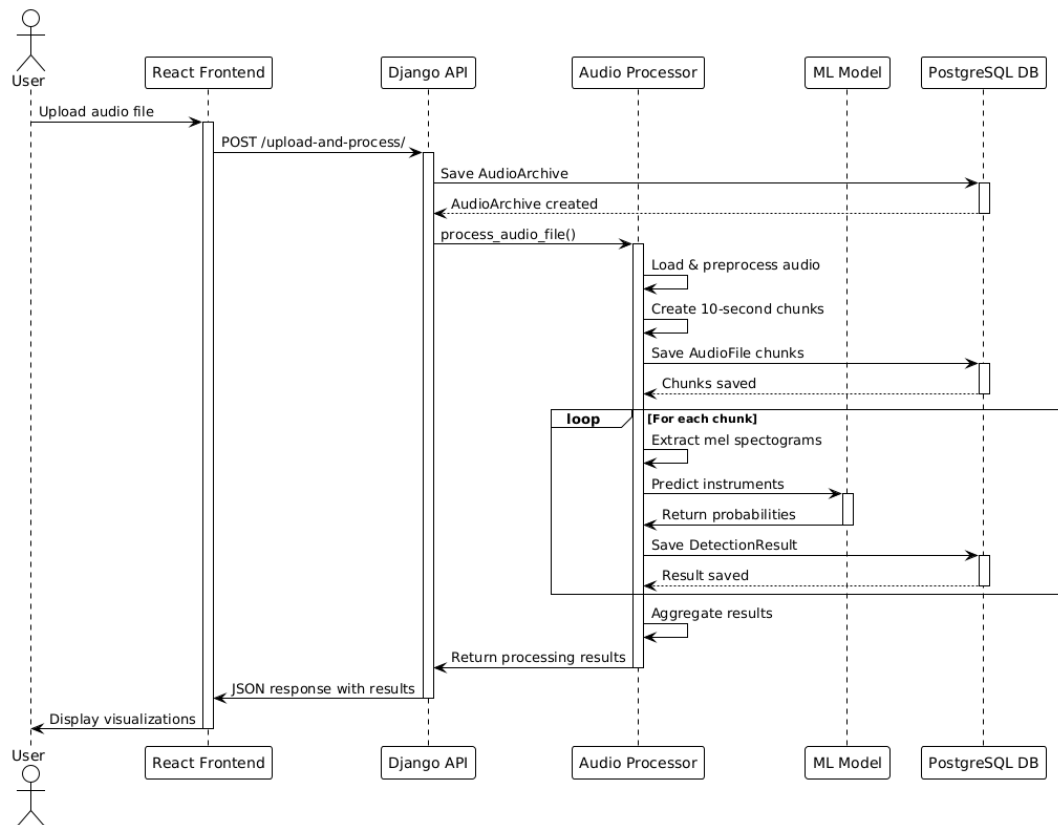


Figure 3. 4: Sequence Diagram of EthnoArchive

The sequence diagram illustrates the interaction between actors in the audio file processing workflow on a single machine. The actors include the User, React Frontend, Django API, Audio Processor, ML Model, and PostgreSQL Database. The process begins with the User uploading an audio file via the React Frontend. The Frontend sends the file to the Django API through a specific endpoint. The Django API validates the file and saves it to local storage, then performs a database operation to store metadata in the PostgreSQL Database. The API then invokes the Audio Processor, which converts the file to a standard format, splits it into chunks, and extracts mel spectograms from each chunk. For each chunk, the Audio Processor sends spectograms to the ML Model for classification, looping through all chunks to collect predictions. The ML Model returns prediction results to the Audio Processor, which aggregates them. The aggregated results are sent back to the Django API, which then sends a JSON response with results to the React Frontend. Finally, the React Frontend displays visualizations to the User.

which stores them in the PostgreSQL. Finally, the Django API returns the results to the React Frontend via a response and the Frontend displays the results to the User.

Activity Diagram

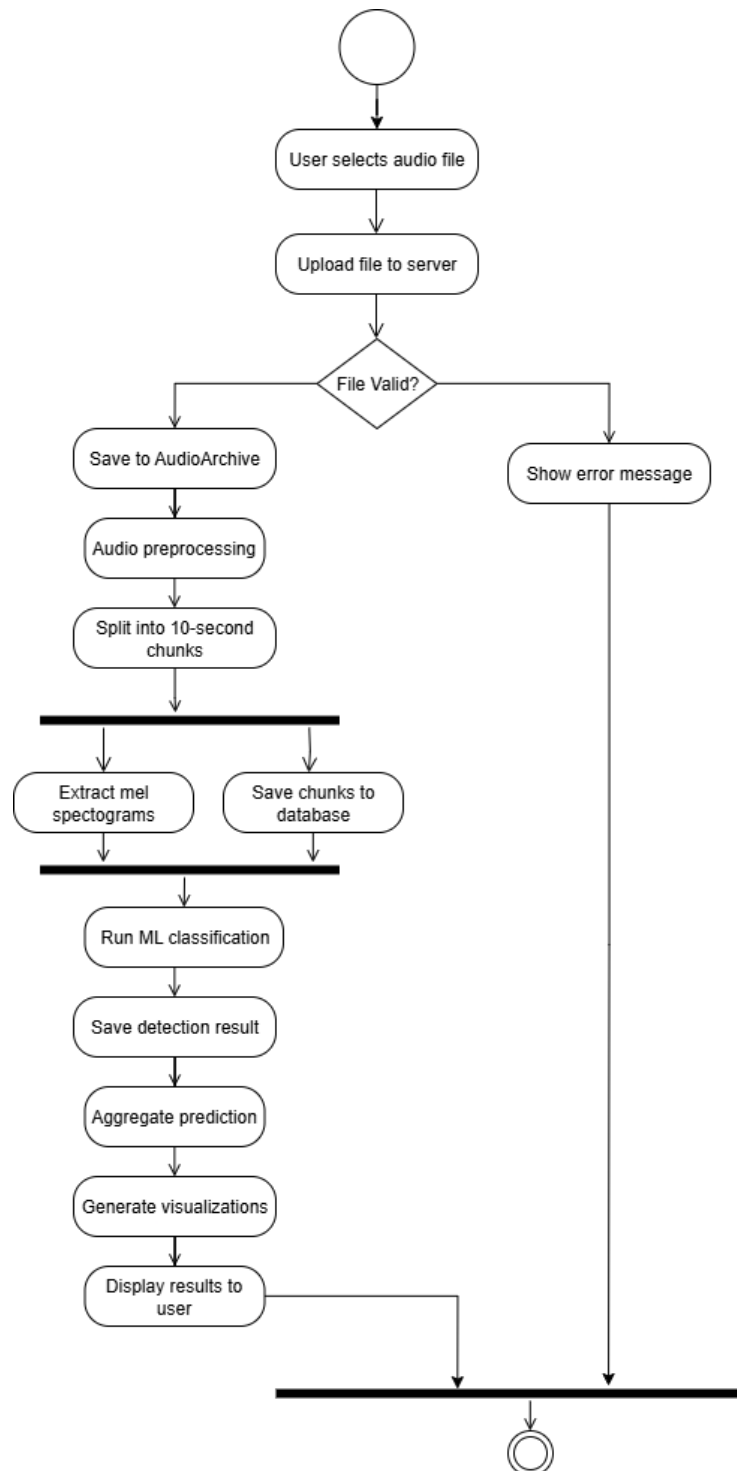


Figure 3. 5: Activity Diagram of EthnoArchive

The activity diagram depicts the audio file processing workflow on a single machine. It begins with the user selecting an audio file, followed by a validation step to check the file's validity. If the file is invalid, an error message is displayed to the user, terminating the process. For valid files, the system proceeds by uploading and saving the file locally, converting it to a standard audio format, and splitting it into smaller chunks. In parallel, the system extracts features from these chunks while simultaneously saving them to a PostgreSQL database. The extracted features are then processed through machine learning classification, and the results are aggregated. Finally, the aggregated results are displayed to the user. The diagram emphasizes concurrent feature extraction and database saving, with a clear error handling path for invalid files.

CHAPTER 4

SYSTEM DESIGN

4.1 Design

The Traditional Musical Instrument Classification Project is designed as a web-based digital archive system that integrates machine learning for automated classification of traditional Nepali instruments (Bansuri, Khin, Madal, Dhimey, and Sarangi) from audio recordings. The system architecture is modular, comprising a frontend for user interaction, a backend for processing and model inference, a database for storage, and an audio processing pipeline for feature extraction. This section outlines the system's architecture, key components, data flow and design considerations.

4.1.1 System Architecture

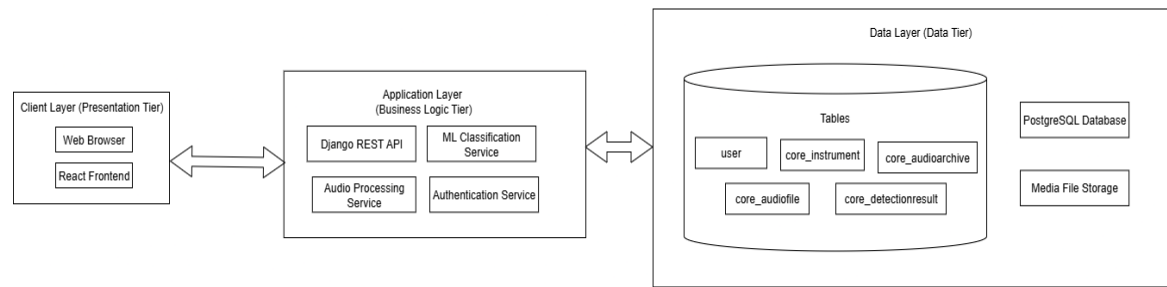


Figure 4. 1: System Architecture of EthnoArchive

The architecture of the Traditional Music Instrument Classification and Archive System follows a three-tier design consisting of the Client Layer, Application Layer, and Data Layer. The Client Layer (Presentation Tier) is the front-facing part of the system, where users interact through a web browser or a React-based frontend interface. This layer communicates with the Application Layer (Business Logic Tier), which contains the core services that power the system. These include the Django REST API for handling client requests, the Machine Learning Classification Service for identifying traditional instruments from uploaded audio, the Audio Processing Service for feature extraction and preprocessing, and the Authentication Service for managing user login and security. The Application Layer, in turn, interacts with the Data Layer (Data Tier), which manages persistent storage. The system uses a PostgreSQL database with tables such as user, core_instrument, core_audioarchive, core_audiofile, and core_detectionresult to store user

details, instrument metadata, audio files, and classification results. Additionally, Media File Storage is used to store the actual audio files uploaded by users. This layered architecture ensures clear separation of concerns: the Client Layer focuses on user interaction, the Application Layer handles business logic and model processing, and the Data Layer manages reliable storage and retrieval of data.

4.1.2 Deployment Diagram

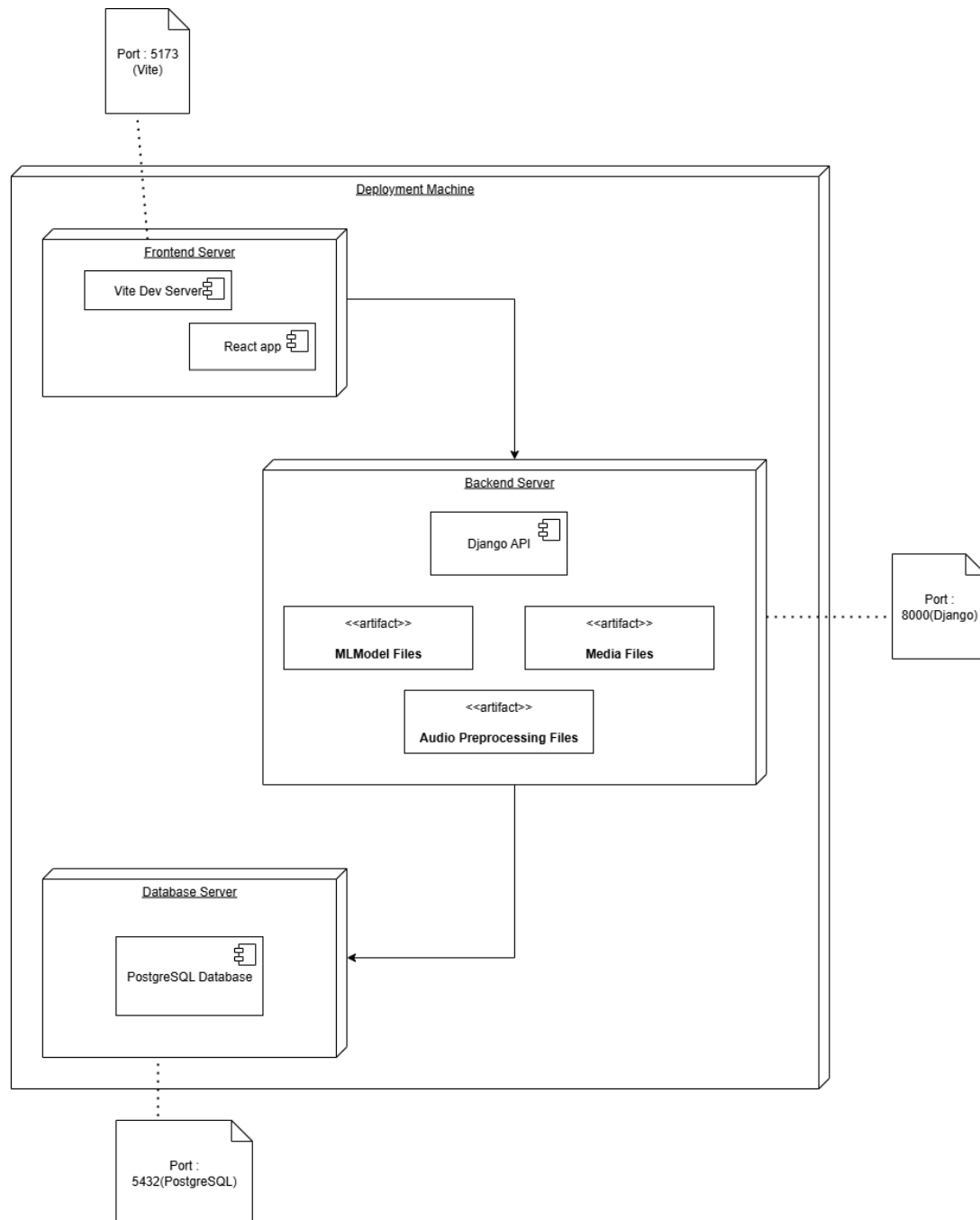


Figure 4. 2: Deployment Diagram of EthnoArchive

The system is deployed on a single machine and follows a three-tier architecture, comprising a presentation tier, application tier, and data tier. The frontend server hosts a React-based user interface served by Vite on port 5173, responsible for rendering the client-side interface and handling user interactions. It communicates with the backend server, which is built using Django and runs on port 8000, serving as the application layer that processes business logic and manages API requests. The data tier consists of a PostgreSQL database running on port 5432, providing persistent storage for application data, while the Django backend also interacts with a local SQLite database for supplementary data management. Additionally, media files and machine learning model files are stored in local file storage, directly accessible by the Django backend. All components operate within the same machine, enabling efficient local communication between the frontend, backend, and database layers.

4.2 Algorithm Description

This section provides a detailed description of the algorithm, its mathematical foundation. The Traditional Musical Instrument Classification Project implements following algorithms:

4.2.1 Fast Fourier Transform (FFT) - Cooley-Tukey Algorithm

FFT enables the transformation of time-domain audio signals into the frequency domain for Mel spectrogram generation. The Cooley-Tukey FFT algorithm, a highly efficient implementation of the DFT, is employed due to its computational efficiency and suitability for real-time spectral analysis.

The Cooley-Tukey FFT algorithm is a divide-and-conquer approach to computing the DFT, which converts a sequence of time-domain samples into their frequency-domain representation. In the context of this project, the FFT is used within the STFT to analyze 10-second audio clips of Nepali traditional instruments. The frequency-domain output is essential for generating Mel spectrograms, which serve as input features for the CNN used in instrument classification.

The algorithm reduces the computational complexity of the DFT from ($O(N^2)$) to ($O(N\log N)$), where (N) is the number of input samples, making it feasible to process audio signals efficiently. The Cooley-Tukey algorithm, specifically its radix-2 variant, requires

the input size (N) to be a power of 2, which is ensured through zero-padding in the audio preprocessing pipeline. The algorithm operates by recursively dividing the DFT into smaller subproblems, combining their results through butterfly operations, and reordering the input data via bit-reversal permutation.

The FFT is applied to short, overlapping frames of the audio signal (typically 25-50 ms) within the STFT framework. Each frame is windowed (e.g., using a Hamming window) to reduce spectral leakage, and the FFT computes the power spectrum, which is then mapped to the Mel scale for spectrogram generation. This process captures the spectral characteristics of the audio, enabling the CNN to distinguish the unique timbral qualities of each instrument.

The Discrete Fourier Transform (DFT) transforms a sequence of N time-domain samples $x[n]$ (where $n=0, 1 \dots N-1$) into a sequence of frequency-domain coefficients $X[k]$ (where $k=0, 1 \dots N-1$). The DFT is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi kn}{N}} \quad 4.1$$

Where $e^{-j\frac{2\pi kn}{N}}$ is the complex exponential (also known as the twiddle factor), and $j=\sqrt{-1}$. The direct computation of the DFT requires $O(N^2)$ operations, which becomes computationally expensive for large values of N , as is common in audio signal processing.

Cooley-Tukey FFT algorithm optimizes this computation by exploiting the symmetry and periodicity of the twiddle factors. In the radix-2 implementation, the input sequence is divided into two smaller sequences: one containing the even-indexed samples $x[2m]$ and the other containing the odd-indexed samples $x[2m+1]$. The DFT of the original sequence is then expressed in terms of the DFTs of these smaller sequences:

$$\begin{aligned} X[k] &= X_{\text{even}}[k] + W_N^k \cdot X_{\text{odd}}[k] \\ X[k+\frac{N}{2}] &= X_{\text{even}}[k] - W_N^k \cdot X_{\text{odd}}[k] \end{aligned} \quad 4.2$$

Where $W_N^k = e^{-j\frac{2\pi k}{N}}$ is the twiddle factor, and $X_{\text{even}}[k]$, $X_{\text{odd}}[k]$ are the $N/2$ -point DFTs of the even and odd indexed subsequences, respectively. This process, known as a butterfly operation, is applied recursively, breaking down the DFT into smaller DFTs until the base case (e.g., a 2-point DFT) is reached.

To ensure correct data ordering during the recursive computation, the algorithm begins with a bit-reversal permutation, which reorders the input sequence based on the reversed binary representation of the indices. For example, for $N=8$, the index 2 (binary: 010) is swapped with index 4 (binary: 100). This reordering allows the butterfly stages to operate in-place and access the correct data pairs efficiently.

The resulting frequency-domain coefficient $X[k]$ represent the amplitude and phase of the underlying frequency components in the signal. These coefficients are used to compute the power spectrum, defined as $|X[k]|^2$, for each audio frame. The power spectra are then mapped onto the Mel scale using a set of triangular filter banks to produce Mel spectrograms. These Mel spectrograms emphasize perceptually relevant frequency information and are widely used as input features for audio classification tasks such as musical instrument recognition.

In this project, the Cooley-Tukey FFT is implemented within the audio processing pipeline to compute the STFT for each audio frame. The FFT's efficiency ensures that the transformation from time-domain audio to frequency-domain spectra is performed rapidly, even for large datasets of 10-second clips. The algorithm's output is critical for generating Mel spectrograms, which are fed into the CNN model to classify instruments. The radix-2 requirement is met by zero-padding audio frames to the nearest power of 2, ensuring compatibility.

```

def _manual_fft(x):
    x = np.asarray(x, dtype=np.complex64)
    n = x.shape[0]
    levels = int(np.log2(n))
    _bit_reverse_traverse(x)

    # Cooley-Tukey butterflies
    for L in range(1, levels+1):
        le = 2**L
        le2 = le//2
        u = np.exp(-2j*np.pi*np.arange(le2)/le)
        for k in range(0, n, le):
            for j in range(le2):
                t = u[j]*x[k+j+le2]
                x[k+j+le2] = x[k+j] - t
                x[k+j] = x[k+j] + t
    return x

```

Figure 4. 3: Manual FFT Algorithm Implementation

4.2.2 STFT

STFT is a time-frequency analysis technique that transforms a time-domain audio signal into a two-dimensional representation, capturing how the frequency content evolves over time. Unlike the standard Fourier Transform, which provides a global frequency spectrum, the STFT divides the signal into short, overlapping frames and applies the FFT to each frame. This process produces a spectrogram, a matrix where one axis represents time, the other represents frequency, and the values indicate the magnitude or power of frequency components.

In this project, the STFT is used to process 10-second audio clips of solo instrument recordings. Each clip is segmented into short frames (typically 25-50 ms), and a window function is applied to reduce spectral leakage. The FFT, implemented via the Cooley-Tukey algorithm, is then computed for each frame to obtain the frequency spectrum. The resulting spectrogram is mapped to the Mel scale to generate Mel spectrograms, which capture perceptually relevant features for instrument classification.

Key parameters in the STFT implementation include the window size, hop length (frame overlap), and window function. The project uses a Hann window to smooth transitions between frames and a 50% overlap (hop length = window length / 2) to ensure continuity in the time-frequency representation.

The STFT of a discrete-time signal $x[n]$ is defined as:

$$STFT(m, \omega) = \sum_{n=-\infty}^{\infty} x[n] \cdot w[n - m] \cdot e^{-j\omega n} \quad 4.3$$

Where,

- $x[n]$ is the input audio signal.
- $w[n-m]$ is the window function (e.g., Hann window) centered at time frame index m .
- ω is the angular frequency (in radians per sample).
- $e^{-j\omega n}$ is the complex exponential used in the Fourier transform.
- m represents the time frame index, determining the position of the sliding window along the signal.

The STFT produces a complex-valued time-frequency representation of the signal, where $STFT(m, \omega)$ encodes the frequency content of the signal around time frame m at frequency

ω . In practice, the magnitude squared of the STFT is computed to obtain the power spectrogram:

$$|STFT(m, \omega)|^2 \quad 4.4$$

This power spectrogram is used as the basis for generating Mel spectrograms, which are perceptually meaningful representations of the audio signal.

The number of time frames in the STFT output is determined by the length of the signal, the window length, and the hop length. For a signal of length N , window length win_len , and hop length hop_len , the number of frames is given by:

$$n_{frames} = 1 + \left\lceil \frac{N - win_len}{hop_len} \right\rceil \quad 4.5$$

In this project, a 50% overlap between consecutive windows is used, which corresponds to setting the hop length to half the window length:

$$hop_len = \frac{win_len}{2} \quad 4.6$$

This ensures smooth transitions between adjacent frames and reduces temporal artifacts.

A Hann window is applied to each frame to minimize spectral leakage, which occurs due to discontinuities at the edges of finite-length signal segments. The Hann window is defined as:

$$w[n] = 0.5 \cdot \left(1 - \cos\left(\frac{2\pi n}{win_len - 1}\right) \right), 0 \leq n \leq win_len \quad 4.7$$

This tapering smoothly reduces the signal amplitude at the frame edges, resulting in cleaner frequency-domain representations.

To compute the Fourier transform efficiently for each windowed frame, the STFT employs the Cooley-Tukey FFT algorithm. This reduces the computational complexity from $O(win_len^2)$ to $O(win_len \log win_len)$ per frame, making real-time or large-scale audio processing feasible.

The resulting power spectrogram is then transformed onto the Mel scale using a bank of triangular filters. This mapping compresses the frequency axis to better match human auditory perception, emphasizing lower frequencies where the ear is more sensitive. The final output, known as mel spectrograms, is used as input to CNN for musical instrument classification.

```

def custom_stft(signal, win_len=WIN_LEN, hop_len=HOP_LEN):
    """Compute the Short-Time Fourier Transform (STFT) of a signal."""
    pad_len = win_len // 2
    padded_signal = np.pad(signal, (pad_len, pad_len), mode='reflect')

    win = hann(win_len)
    n_frames = 1 + (len(padded_signal)-win_len)//hop_len
    fft_len = nextpow2(win_len)
    spec = np.zeros((n_frames, fft_len//2+1), dtype=np.complex128)

    for t in range(n_frames):
        start = t * hop_len
        end = start + win_len
        chunk = padded_signal[start:end] * win
        fft = manual_rfft(chunk, fft_len)
        spec[t] = fft
    return spec.T

```

Figure 4. 5: Custom STFT Implementation

```

def hann(N):
    return 0.5 - 0.5*np.cos(2*np.pi*np.arange(N)/(N-1))

```

Figure 4. 4: Hann window implementation

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation

The Nepali Traditional Instrument Classification Project integrates a machine learning-based audio classification system with a web application to create a digital archive for Nepali traditional instruments. The implementation encompasses the audio processing pipeline, CNN model, web frontend and backend, and database, all developed in a local environment. This section details the implementation process.

5.1.1 Tools Used

Programming Languages

- Python : Backend development, machine learning, audio processing
- TypeScript : frontend development with type safety
- JavaScript : Frontend logic and interactions

Web Development Frameworks

- Backend Development: Djanggo, Django REST Framework
- Frontend Development: React

Machine Learning and Audio Processing

- Tensorflow: Deep learning model development and inference
- Librosa: Audio analysis
- Audacity: Audio processing
- Pydub: Audio file manipulation and format conversion

Database and Storage

- PostgreSQL: Primary database
- DjangoORM: Object-relational mapping
- Local file system for file storage

Development Environment

- Visual Studio Code: Primary code editor
- Git/Github: Version control

- Google Collab: ML model training

5.1.2 Data Preprocessing

The data preprocessing pipeline is a critical component of the instrument classification system that transforms raw audio files into standardized, feature-rich representations suitable for machine learning. This section details the complete preprocessing workflow, including validation, normalization, conversion, chunking, and feature extraction processes.

i. Input data validation

This stage ensures that uploaded audio files meet the system's requirements for format, size, and integrity, preventing errors during downstream processing. By validating inputs, the system guarantees compatibility with the audio processing pipeline and protects against corrupt or unsupported files, which is particularly important for user-uploaded content from diverse sources.

- **File Format Validation:** Checks the file extension against a predefined set of supported audio formats, including MP3, M4A, AAC, FLAC, OGG, OPUS, and WAV. This ensures compatibility with standard audio processing libraries.
- **File Size Validation:** Enforces a maximum file size of 100 MB to manage computational resources and prevent excessive memory usage.
- **File Integrity Check:** Verifies that the file is not corrupted by attempting to load it, ensuring it can be processed without errors.

```
# Check file extension
ext = uploaded_file.name.split(".")[-1].lower()
if ext not in ALLOWED_EXTENSIONS:
    raise ValidationError(
        f"Unsupported file type '{ext}'. Allowed: {'', ' '.join(ALLOWED_EXTENSIONS)}"
    )

# Check file size (limit to 100MB)
max_size = 100 * 1024 * 1024 # 100MB
if uploaded_file.size > max_size:
    raise ValidationError(f"File too large. Maximum size: {max_size / (1024*1024):.1f}MB")
```

Figure 5. 1: File Validation Implementation

ii. Audio Format Standardization

To ensure consistent processing, all audio files are converted to a standardized format with uniform specifications. This step addresses variability in sample rates, channels, and bit depths, which is common in audio sourced from public platforms, enabling seamless integration with the audio processing pipeline. Converts audio to WAV format with PCM encoding, using a sample rate of 22,050 Hz (22.05 kHz), mono channel, and 16-bit depth.

```
def convert_to_wav(audio_segment_or_path):
    audio = AudioSegment.from_file(audio_segment_or_path)

    # Convert to target specifications
    audio = (
        audio
        .set_frame_rate(22050)      # 22.05 kHz
        .set_channels(1)            # mono
        .set_sample_width(2)        # 16-bit (2 bytes)
    )

    # Export to WAV format
    wav_buffer = BytesIO()
    audio.export(wav_buffer, format="wav", codec="pcm_s16le")
    return wav_buffer
```

Figure 5. 2: Audio Format Standardization Implementation

iii. Audio Preprocessing

This stage prepares standardized audio for feature extraction by normalizing amplitude. Scales the audio signal to a range of [-1, 1] by dividing each sample by the maximum absolute amplitude. This preserves relative amplitude relationships while preventing clipping and ensuring consistent dynamic range.

```
def normalize_audio(audio):
    if np.max(np.abs(audio)) > 0:
        audio = audio / np.max(np.abs(audio))
    return audio
```

Figure 5. 3: Audio Normalization Implementation

iv. Audio Segmentation (Chunking)

Audio files are divided into fixed 10-second segments to ensure uniform input for the CNN model. This stage handles variable-length audio by creating non-overlapping chunks, padding short segments, and skipping excessively short ones.

```
def _create_audio_chunks(self, audio_archive, audio_data):
    chunk_ms = self.chunk_duration * 1000 # 10,000 ms
    min_ms = self.min_chunk_duration * 1000 # 8,000 ms

    for i, start in enumerate(range(0, len(audio_data), chunk_ms)):
        chunk = audio_data[start:start + chunk_ms]

        # Skip very short chunks
        if len(chunk) < min_ms:
            continue

        # Pad short chunks with silence
        if len(chunk) < chunk_ms:
            silence_needed = chunk_ms - len(chunk)
            chunk += AudioSegment.silent(duration=silence_needed)
```

Figure 5. 4: Chunking Implementation

v. Mel Spectrogram Extraction

This stage extracts Mel spectrograms from segmented audio, providing a time-frequency representation that captures the spectral characteristics of each instrument. Mel spectrograms are used as input features for the CNN, leveraging human auditory perception for effective classification.

```
def wav_to_logmelspec_original(signal, sr=22050):
    spec = custom_stft(signal)
    power = (np.abs(spec))**2
    fb = librosa.filters.mel(sr=sr, n_fft=WIN_LEN, n_mels=N_MELS, fmin=0, fmax=sr/2)
    mel = fb @ power
    logmel = np.log(mel + 1e-6)
    logmel_norm = (logmel - np.mean(logmel)) / (np.std(logmel) + 1e-8)
    return logmel_norm.T
```

Figure 5. 5: Mel Spectrogram Extraction Implementation

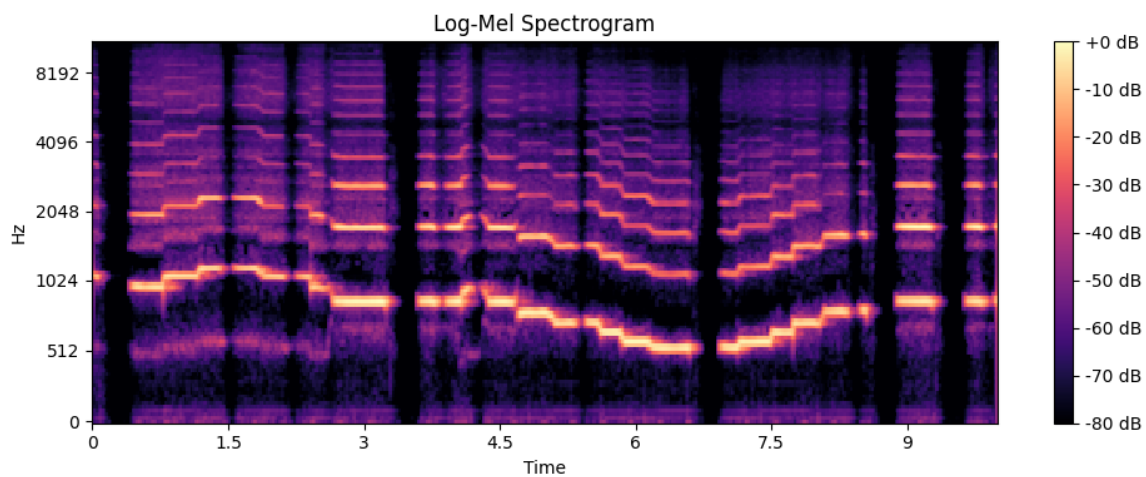


Figure 5. 8: Bansuri Mel Spectrogram Visualization

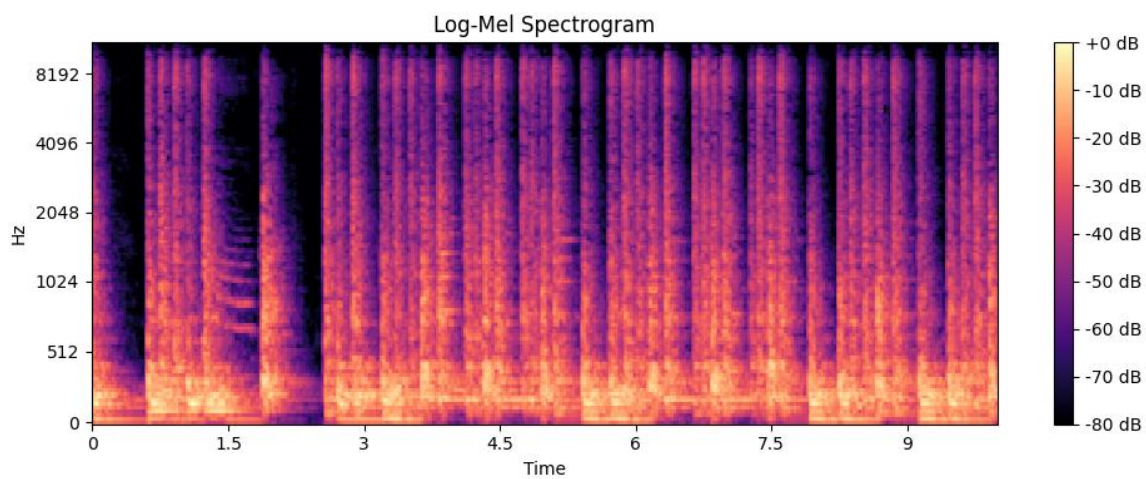


Figure 5. 7: Dhimey Mel Spectrogram Visualization

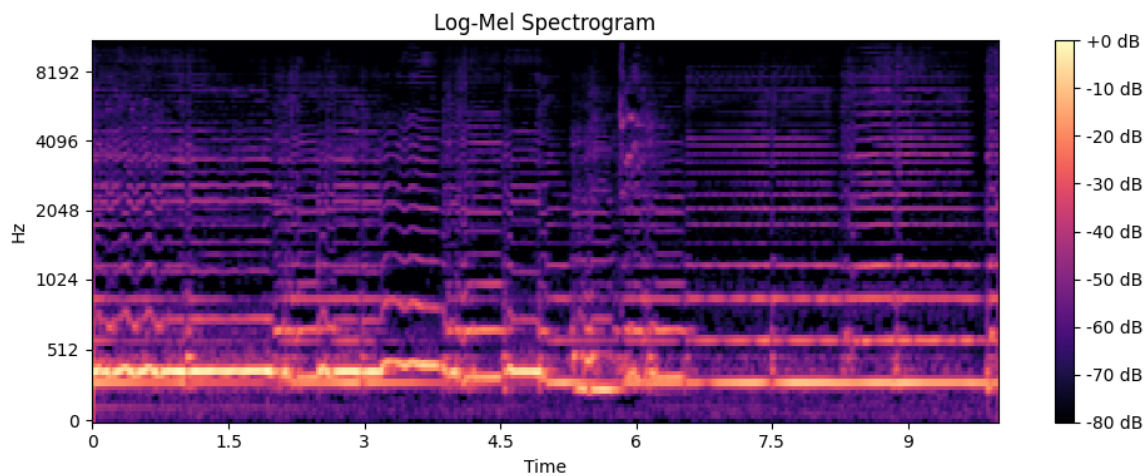


Figure 5. 6: Sarangi Mel Spectrogram Visualization

5.1.3 Data Preparation

The dataset is organized into train, validation, and test splits, stored in a directory structure with corresponding .npy files (Mel spectrograms) and CSV files containing one-hot encoded labels for the five instrument classes. The data is loaded using a custom function that validates file paths, ensures correct spectrogram shapes (431, 128, 1), and tracks class distributions. The dataset is split into 70% training, 15% validation, and 15% test sets, with class weights applied to handle imbalance during training.

	filename	dhimey	sarangi	bansuri	madal	khin
0	madal_228	0	0	0	1	0
1	madal_21	0	0	0	1	0
2	dhimey_33	1	0	0	0	0
3	bansuri_351	0	0	1	0	0
4	bansuri_97	0	0	1	0	0
5	bansuri_207	0	0	1	0	0
6	sarangi_20	0	1	0	0	0
7	bansuri_156	0	0	1	0	0
8	bansuri_khii_166	0	0	1	0	1
9	sarangi_147	0	1	0	0	0

Figure 5. 9: CSV File Format Demo

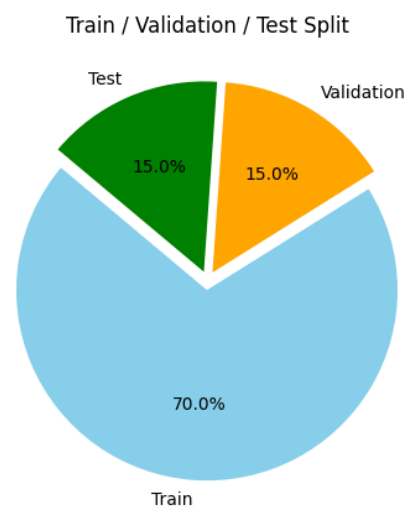


Figure 5. 10: Data Split Summary

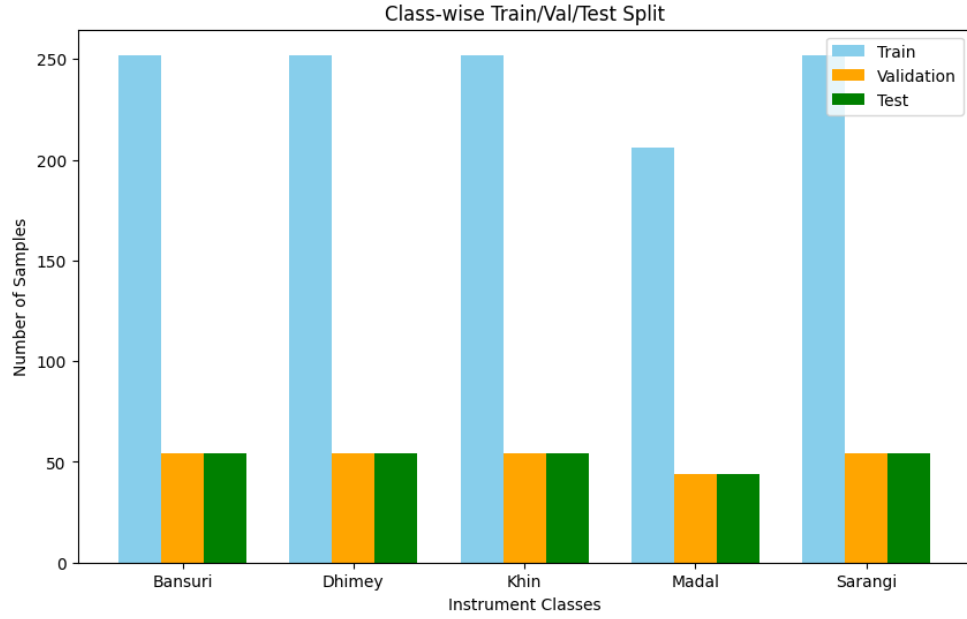


Figure 5. 11: Classwise Distribtuition Summary

5.1.4 Model Architecture

The machine learning model for the Nepali Traditional Instrument Classification Project is a CNN designed to classify Mel spectrograms of 10-second audio clips into one of five instrument classes: Bansuri, Khin, Madal, Dhimey, and Sarangi. The model is implemented using TensorFlow.

The CNN model processes input Mel spectrograms with a shape of (431, 128, 1), representing 431 time frames, 128 Mel frequency bands, and a single channel (mono). The architecture consists of a sequence of convolutional, pooling, normalization, and dense layers, designed to balance feature extraction and computational efficiency. The key components are:

Input Layer:

Accepts Mel spectrograms with dimensions (431, 128, 1). The input shape corresponds to the time-frequency representation generated by the audio preprocessing pipeline, capturing the spectral and temporal characteristics of the audio.

Convolutional Layers:

- **First Convolutional Block:** A 2D convolutional layer with 32 filters, a (3, 3) kernel, ReLU activation, and L2 regularization (coefficient 0.01) to prevent overfitting. Same padding ensures the output dimensions match the input. This is followed by batch

normalization to stabilize training, a max-pooling layer (2, 2) to reduce spatial dimensions, and a dropout layer (0.4) for regularization.

- **Second Convolutional Block:** A convolutional layer with 64 filters, (3, 3) kernel, ReLU activation, and L2 regularization (0.01), followed by batch normalization, max-pooling (2, 2), and dropout (0.5). This block extracts higher-level features.
- **Third Convolutional Block:** A convolutional layer with 128 filters, (3, 3) kernel, ReLU activation, and L2 regularization (0.01), followed by batch normalization, max-pooling (2, 2), and dropout (0.6). This block captures complex patterns in the spectrogram.

Global Pooling:

A global average pooling layer reduces the spatial dimensions to a single vector, summarizing the features extracted by the convolutional layers. This reduces computational complexity and prevents overfitting by eliminating the need for large fully connected layers.

Dense Layers:

- **First Dense Layer:** A fully connected layer with 128 units, ReLU activation, and L2 regularization (0.01), followed by batch normalization and dropout (0.7). This layer integrates features for classification.
- **Second Dense Layer:** A fully connected layer with 64 units, ReLU activation, and L2 regularization (0.01), followed by batch normalization and dropout (0.6). This layer refines the feature representation.
- **Output Layer:** A dense layer with 5 units (one per instrument class) and sigmoid activation, producing class probabilities for multi-label classification. The sigmoid activation allows the model to handle cases where an audio clip might theoretically contain multiple instruments, though the dataset focuses on solo recordings.

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 431, 128, 1)	0
conv2d_12 (Conv2D)	(None, 431, 128, 32)	320
batch_normalization_20 (BatchNormalization)	(None, 431, 128, 32)	128
max_pooling2d_12 (MaxPooling2D)	(None, 215, 64, 32)	0
dropout_20 (Dropout)	(None, 215, 64, 32)	0
conv2d_13 (Conv2D)	(None, 215, 64, 64)	18,496
batch_normalization_21 (BatchNormalization)	(None, 215, 64, 64)	256
max_pooling2d_13 (MaxPooling2D)	(None, 107, 32, 64)	0
dropout_21 (Dropout)	(None, 107, 32, 64)	0
conv2d_14 (Conv2D)	(None, 107, 32, 128)	73,856
batch_normalization_22 (BatchNormalization)	(None, 107, 32, 128)	512
max_pooling2d_14 (MaxPooling2D)	(None, 53, 16, 128)	0
dropout_22 (Dropout)	(None, 53, 16, 128)	0
global_average_pooling2d_4 (GlobalAveragePooling2D)	(None, 128)	0
dense_12 (Dense)	(None, 128)	16,512
batch_normalization_23 (BatchNormalization)	(None, 128)	512
dropout_23 (Dropout)	(None, 128)	0
dense_13 (Dense)	(None, 64)	8,256
batch_normalization_24 (BatchNormalization)	(None, 64)	256
dropout_24 (Dropout)	(None, 64)	0
dense_14 (Dense)	(None, 5)	325

Total params: 119,429 (466.52 KB)
 Trainable params: 118,597 (463.27 KB)
 Non-trainable params: 832 (3.25 KB)

Figure 5. 12: Model Architecture

5.2 Testing

Table 5. 1 Test Cases for authentication module

Test Case ID	Test Scenario	Steps	Expected Result	Actual Result	Status
AUTH-01	Register new user with valid data	Enter valid username, email, password and then click Create Account	User account created and is redirected to login	User account is created successfully and redirected to login page	Pass
AUTH-02	Register with existing email	Enter already registered email then click Create Account	Error message: "Username already exists"	"Username already exists" error message is displayed	Pass
AUTH-03	Register with different password retype	Enter valid username, email, password and type different password in confirm password field	Error message: "Passwords do not match"	"Passwords do not match" error message is displayed	Pass
AUTH-04	Login with correct credentials	Enter valid username/password then click Sign In	User logged in and redirected to homepage	User is logged in and redirected to the homepage	Pass
AUTH-05	Login with incorrect password	Enter wrong password	Error message: "Invalid credentials"	"Invalid Credentials" error message is displayed	Pass

AUTH-06	Access upload without login	Try uploading file without logging in	Redirect to login page	User is redirected to login page	Pass
AUTH-07	Login with blank fields	Do not enter anything. Then press Sign In button.	“Username is required” and “Password is required” error message should be displayed.	User is displayed required error messages	Pass

Table 5. 2 Test Cases for upload and processing module

Test Case ID	Test Scenario	Steps	Expected Result	Actual Result	Status
UP-01	Upload valid audio file	Select valid .wav or .mp3 file and click Analyze Instrument button	File should be uploaded and start processing	The file is uploaded and processing starts	Pass
UP-02	Upload unsupported file type	Try uploading .txt or .jpg	Error message: "Please select a valid audio file (MP3, WAV, FLAC, OGG, M4A, AAC)"	"Please select a valid audio file (MP3, WAV, FLAC, OGG, M4A, AAC)" error message is displayed	Pass

UP-03	Upload large file	Try uploading file> allowed limit	Error message: "File too large. Maximum size:100MB"	"File too large. Maximum size:100MB" error message is displayed	Pass
UP-04	Upload and verify progress bar	Upload valid file	Progress bar should shown until completed	Progress bar is shown until the file is processed	Pass
UP-05	Upload by unauthorized user	Try uploading file without logging in	Redirect to login	User is redirected to login page	Pass

Table 5. 3 Instrument Detection Module

Test Case ID	Test Scenario	Steps	Expected Result	Actual Result	Status
DET-01	Detect instrument from valid sarangi audio	Upload clear instrument audio	System predicts correct instrument and save in archive with metadata in db	System predicted Sarangil with 94% confidence. Entry created in DB with correct mapping.	Pass
DET-02	Detect instrument from valid dhimey audio	Upload clear instrument audio	System predicts correct instrument and save in archive with metadata in db	System predicted Dhimey with 95% confidence. Entry created in DB with correct mapping.	Pass

DET-03	Detect instrument from valid bansuri audio	Upload clear instrument audio	System predicts correct instrument and save in archive with metadata in db	System predicted Bansuri with 88% confidence. Entry created in DB with correct mapping.	Pass
DET-04	Detect instrument from valid madal audio	Upload clear instrument audio	System predicts correct instrument and save in archive with metadata in db	System predicted Madal with 81% confidence. Entry created in DB with correct mapping.	Pass
DET-05	Detect instrument from valid khin audio	Upload clear instrument audio	System predicts correct instrument and save in archive with metadata in db	System predicted Sarangi with 60% confidence. Entry created in DB with correct mapping.	Pass
DET-06	Low confidence prediction	Upload noisy or unclear sarangi audio	System shows prediction with low confidence	System predicted Khin with 88% confidence. Entry created in DB with correct mapping.	Pass
DET-07	Detect when multiple instruments present	Upload mixed audio of Dhimey+Bansuri	System predicts dominant instrument	System predicted Dhimey with 88% confidence. Entry created in DB with correct mapping.	Pass

DET-08	No instrument detected	Upload random noise or audio with silence	Message: "No Instrument Detected"	User is displayed "No Instrument Detected" message	Pass
DET-09	Long audio detection	Upload valid audio of 30 secs	System chunks to 10s, predicts correct instrument	Audio is chunked to 3 samples of 10 secs duration and predicted correct instrument	Pass

5.3 Result Analysis

The classification system for traditional musical instruments shows strong effectiveness, achieving a test accuracy of 92.69% with a low test loss of 0.239. These results indicate solid learning capability and reliable generalization when applied to previously unseen audio samples. A weighted average F1-score of 0.93 confirms balanced precision and recall across all five instrument categories, demonstrating robust classification without strong bias toward any specific class.

Classification Report:				
	precision	recall	f1-score	support
bansuri	1.00	0.91	0.95	54
dhimey	0.82	1.00	0.90	54
khin	1.00	0.81	0.90	54
madal	1.00	0.93	0.96	44
sarangi	0.88	0.98	0.93	54
accuracy			0.93	260
macro avg	0.94	0.93	0.93	260
weighted avg	0.94	0.93	0.93	260

Figure 5. 13 Classification Report

Distinct trends emerge for each instrument category. Bansuri, khin, and madal achieve perfect precision scores of 1.00, meaning classifications for these instruments are always

correct. However, recall varies, with bansuri at 0.91, khin at 0.81, and madal at 0.93, indicating occasional missed detections despite perfect precision.

Dhimey presents a different pattern, achieving recall of 1.00 but precision of 0.82. Every dhimey sample is detected, yet some non-dhimey sounds are incorrectly classified into this category. This behavior suggests that dhimey functions as a frequent fallback class, likely due to dominant percussive patterns within the feature space. Sarangi delivers balanced results with recall of 0.98 and precision of 0.88, combining minimal false negatives with limited false positives.

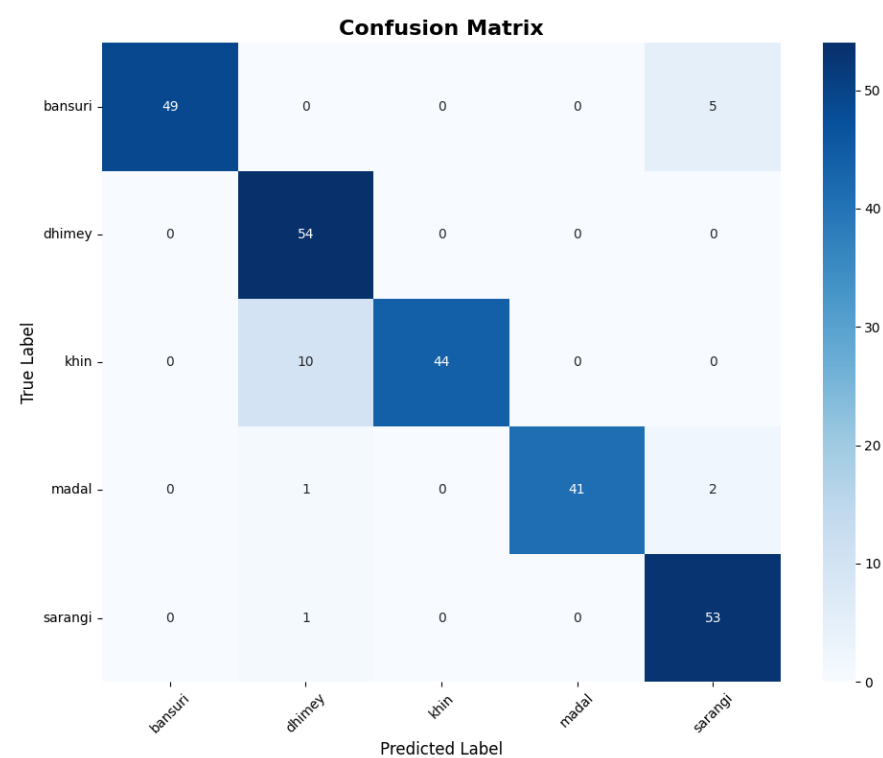


Figure 5. 14 Confusion Matrix

The confusion matrix highlights specific misclassification patterns. Dhimey achieves perfect recall with all 54 samples correctly identified, yet still attracts misclassifications from other instruments. In particular, 10 khin samples, along with single instances from madal and sarangi, are mistakenly labeled as dhimey, reducing its precision despite flawless recall.

The most frequent misclassification occurs between khin and dhimey, with 10 out of 54 khin samples classified as dhimey. This overlap suggests shared acoustic properties in the frequency spectrum that limit differentiation. Bansuri also shows confusion with sarangi, where five bansuri samples are misclassified. Remaining errors are minimal: madal

achieves 41 correct identifications out of 44, and sarangi reaches 53 correct predictions out of 54, reflecting overall strong reliability.



Figure 5. 15 Error Rate by Class

Error rate distribution varies noticeably across instruments. Khin displays the highest error rate at 19%, meaning nearly one out of every five khin samples is misclassified. Bansuri follows with a 9% error rate, while madal maintains a moderate 7%. In contrast, dhimey achieves flawless classification with 0% error, highlighting exceptional recognition accuracy for this percussion instrument. Sarangi records only a 2% error rate, further confirming its highly reliable classification performance.

Differences in error rates suggest that certain instruments possess more distinctive acoustic signatures, making them easier to identify. Dhimey’s perfect performance implies that rhythmic and percussive features are effectively captured by the chosen feature extraction technique. Conversely, higher error rates for instruments such as khin and bansuri point toward challenges in distinguishing tonal subtleties or breath-related acoustic features.

CHAPTER 6

CONCLUSION AND FUTURE RECOMMENDATIONS

6.1 Conclusion

The Traditional Musical Instrument Classification Project has been quite rewarding. The goal was to create something that could help preserve Nepal's rich musical traditions, and the result is a working system that does exactly that. The web application can automatically identify five traditional Nepali instruments: Bansuri, Khin, Madal, Dhimey, and Sarangi from audio recordings using machine learning.

The heart of the system is a neural network that analyzes audio through Mel spectrograms, which are essentially visual representations of sound. The whole thing was built with modern web technologies: a React frontend, a Django backend that handles all the processing, and a PostgreSQL database that stores everything. When someone uploads an audio clip, the system processes it and tells them which instrument they're listening to.

The results have been encouraging. The model achieves about 77% accuracy overall, with nearly perfect discrimination between different instrument types. Some instruments like the Khin and Sarangi are identified correctly over 90% of the time, which is fantastic. However, there are still struggles with the Madal, which indicates a need for more training data for that particular instrument or that it shares similar acoustic features with others.

6.2 Future Recommendations

The most obvious next step would be getting more audio recordings, especially for instruments like the Madal that the system struggles with. Partnerships with Nepali musicians and cultural institutions could help build a more comprehensive dataset. There are also some interesting technical challenges that could be tackled, like making the system work with live audio streams or handling recordings that have multiple instruments playing together.

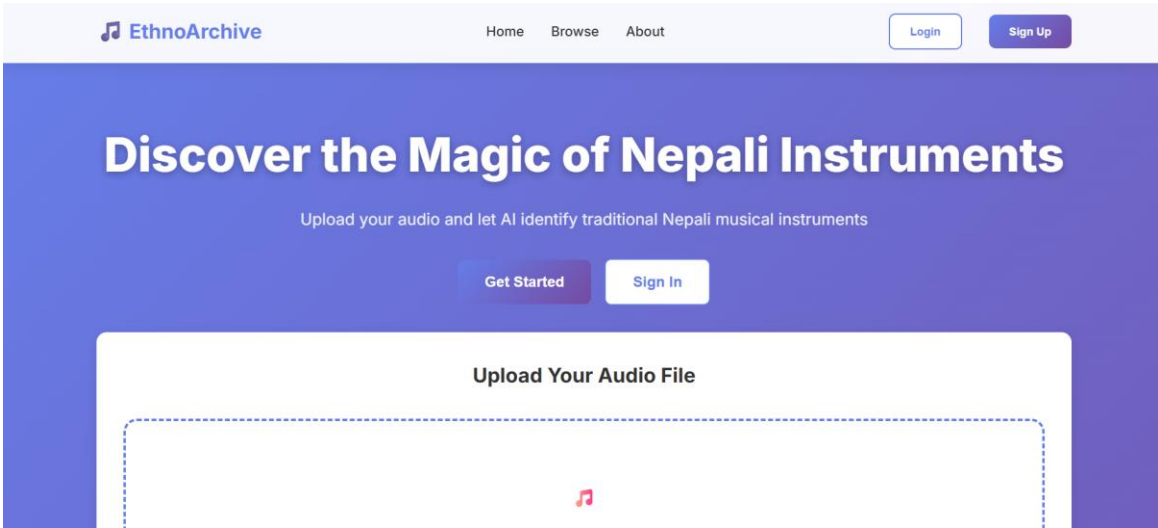
From a technical standpoint, experimentation with more advanced neural network architectures or even using pre-trained models that already understand general audio patterns could be beneficial. Cloud deployment is definitely on the roadmap too, getting this system online where anyone can access it would be a game-changer for reaching a global audience interested in Nepali music.

REFERENCE

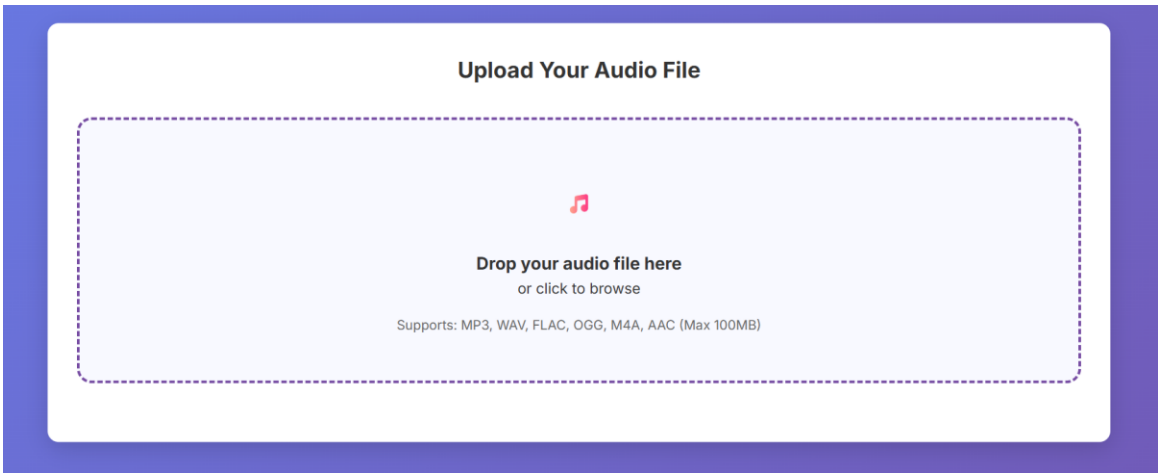
- [1] J. Chulev, "Improving Musical Instrument Classification with Advanced Machine Learning Techniques," 2024.
- [2] L. Haidar-Ahmad, "Music and Instrument Classification using Deep," 2019.
- [3] D. S. Hing and C. J. Settle, "Detecting and Classifying Musical Instruments with Convolutional Neural Networks," Stanford University, Department of Computer Science.
- [4] Z. Liu, "Audio Feature Extraction and Classification Technology Based on Convolutional Neural Network," 2024.
- [5] D. B. B. Ivan Damnjanovic, "ENABLING ACCESS TO SOUND ARCHIVES THROUGH INTEGRATION, ENRICHMENT".
- [6] J. D. Deng, C. Simmermacher and S. Cranefield, "A Study on Feature Analysis for Musical Instrument Classification".

APPENDICES

Home Page



Audio Upload Section



Sign Up page



Join EthnoArchive

Create your account to start analyzing instruments

Username

Email

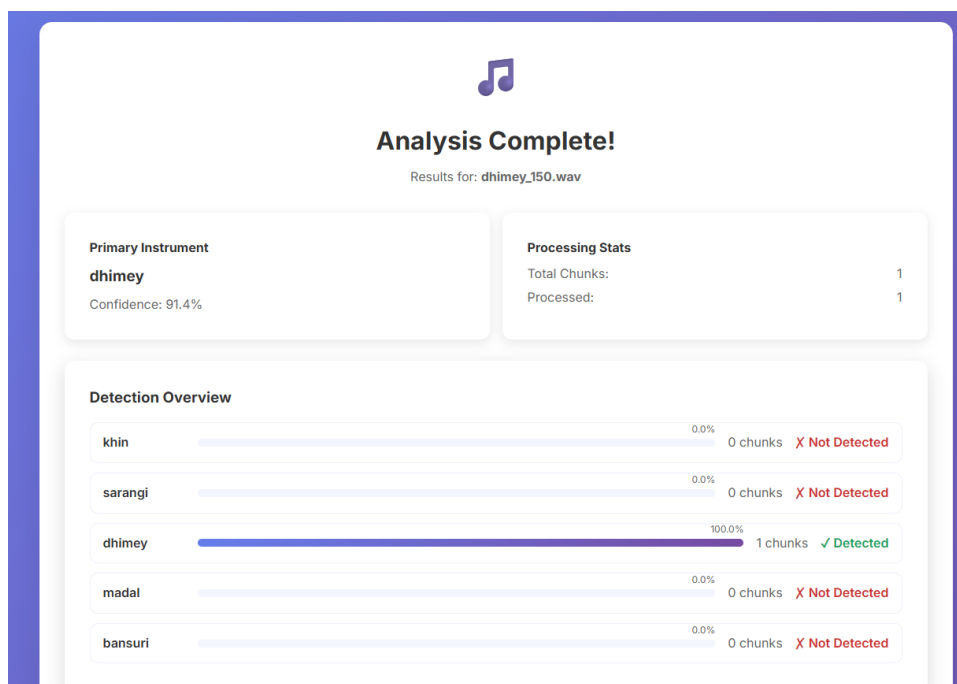
Password

Confirm Password

Create Account

Already have an account? [Sign in here](#)

Results Page



Profile Section

N

@niraj

Music Enthusiast • Instrument Explorer

33

Uploads

31

Detections

5

Instruments

70%

Avg Confidence

Your Music Journey

dhimey

Favorite Instrument

70%

Success Rate

31

Total Detections

Sort your uploads:

Date ↓

Instrument

Confidence

33 uploads

@niraj

9/2/2025, 1:40:26 PM

▶ 0:00 / 0:10

🔊 ⋮

Detected Instrument

dhimey (91%)

@niraj

9/2/2025, 10:22:08 AM

▶ 0:00 / 0:10

🔊 ⋮

Detected Instrument

No clear detection

@niraj

9/2/2025, 10:14:55 AM

▶ 0:00 / 0:31

🔊 ⋮

Detected Instrument

sarangi (67%)

Browse Archive Section

Browse Archives

Sort by:

Date ↓

Instrument

Uploader

30 items

@niraj

9/2/2025, 1:40:26 PM

▶ 0:00 / 0:10

🔊 ⋮

Detected Instrument

dhimey (91%)

All Instruments

dhimey · 91%

@niraj

9/2/2025, 10:22:08 AM

▶ 0:00 / 0:10

🔊 ⋮

Detected Instrument

No clear detection

@niraj

9/2/2025, 10:14:55 AM

▶ 0:00

🔊 ⋮

Detected Instrument

sarangi (67%)

All Instruments

sarangi · 67%

@niraj

9/2/2025, 10:14:21 AM

▶ 0:00 / 0:10

🔊 ⋮

Detected Instrument

sarangi (94%)

All Instruments

sarangi · 94%

@niraj

9/2/2025, 10:13:38 AM

▶ 0:00 / 0:10

🔊 ⋮

Detected Instrument

dhimey (96%)

All Instruments

dhimey · 96%

@niraj

9/2/2025, 10:13:08 AM

▶ 0:00 / 0:10

🔊 ⋮

Detected Instrument

khin (88%)

All Instruments

khin · 88%

Login Page



Welcome Back

Sign in to your account to continue

Username

Password

Sign In

Don't have an account? [Sign up here](#)